

Atividade 5 - Implementação

Link do Repositório

GitHub: <https://github.com/GiovaneVitorCarbonera/SGTerminal>

Descrição da Implementação

O sistema foi desenvolvido como uma API REST em Node.js usando Express, com organização em camadas e acesso ao banco de dados PostgreSQL. A autenticação é feita com JWT, com proteção das rotas por middleware de segurança e controle de permissões. A documentação da API foi estruturada com Swagger.

Estrutura do Backend

- **Controllers:** a lógica principal das requisições está concentrada nas rotas.
- **Services:** DBService.js centraliza a conexão com o PostgreSQL.
- **Models:** as entidades ficam representadas pelas tabelas accounts e veículos no banco.
- **Routes:** auth.js, accounts.js e vehiclemovements.js.
- **Middleware:** auth.js e permissions.js fazem autenticação e autorização.

CRUDs Desenvolvidos

Integrante	Entidade	Operações
Giovane Vitor Carbonera	Accounts / Login	CREATE, READ, UPDATE, DELETE, LOGIN
Romulo Moreira Moreto Guisso	Vehicle Movements	CREATE, READ, UPDATE, DELETE

Cada módulo possui as quatro operações REST completas. O módulo de Login gera o token de acesso e o módulo de Accounts trabalha com as permissões do usuário.

Funcionalidades Implementadas

- Login com geração de JWT.
- Autorização por permissões em rotas protegidas.
- CRUD de accounts com validação de dados e retorno de erros apropriado.
- CRUD de vehicle movements com integração ao PostgreSQL.
- Documentação da API com Swagger.

Banco de Dados

- **Tipo:** PostgreSQL.
- **Estrutura:** tabela accounts com username, password_hash, permissions e created_at; tabela veiculos com placa, datahora, tipo e idaccount.
- **Permissões:** enum account_permission para controle de acesso.

Autenticação

- Tipo: JWT com Bearer Token.
- Funcionamento: o endpoint /auth/login valida usuário e senha, retorna um token com validade de 1 hora e as rotas protegidas usam express-jwt.
- Autorização: permissions.js verifica se o token possui a permissão exigida antes de liberar a requisição.

Testes Realizados

Validação manual dos endpoints da API, usando Swagger.

Dificuldades Encontradas

- Falta de requisitos claros por parte do cliente, gerando incertezas sobre o escopo e os objetivos do projeto.
- Dificuldade na comunicação e na tomada de decisões, causada pela ausência de definições claras e pelas constantes mudanças de direcionamento.
- Necessidade de reformular o projeto três vezes devido à descoberta progressiva dos requisitos, impactando prazos e o andamento do desenvolvimento.

Considerações

O projeto foi feito sem o uso de Models e Controllers por falta de necessidade.